

Pyramid — Cheat Sheet

Pyramid 2.1 (2026-03-11) · Python 3.10+ · WSGI, synchronous
pyramid_tm 2.6 · pyramid_jinja2 2.10 · debugtoolbar 4.12 · pyramid_retry 2.1 · WebTest 3.0
versions verified 2026-10-07 · docs.pylonsproject.org · irurueta.com

SINGLE-FILE APP & PROJECT

```
pip install pyramid waitress
from pyramid.config import Configurator
from pyramid.response import Response
from waitress import serve
def hello(request):
    return Response("Hi ~ + request.matchdict["n"])
with Configurator() as config:
    config.add_route("hello", "/hi/{n}")
    config.add_view(hello, route_name="hello")
    app = config.make_wsgi_app()
    serve(app, listen="127.0.0.1:6543") # WSGI app

cookiecutter gh:PyLons/pyramid-cookiecutter-starter \
  --checkout 2.1-branch # jinja2|chameleon|mako + backend
pip install -e ".[testing]" # pyproject.toml, not setup.py
pserve development.ini --reload # hupper

Layout: development|production|testing.ini pyproject.toml
pkg/ __init__ routes, views, templates, static/ tests/. Entry
point main(global_config, **settings) → Configurator →
make_wsgi_app(). Generated requires-python says 3.8: Pyramid
2.1 needs 3.10+. No async views.
```

CONFIGURATOR, SCAN & INCLUDE

```
config.scan() # venusian: @view_config etc.
config.include("pkg.api", route_prefix="/api")
def includeme(config): config.add_route("b", "/b")
config.add_directive("add_thing", fn) # custom directive
config.action(discriminator, callable) # deferred action
config.commit() # resolve conflicts now

Two phases: calls queue actions → at commit, conflicts are detected;
the includer wins over includees, same-module conflicts raise
ConfigurationConflictError. autocommit=True skips this.
config.registry.introspector lists everything;
ApplicationCreated fires after make_wsgi_app().
```

.INI SETTINGS & PYRAMID *

```
[app:main]
use = egg:bookshelf
pyramid.includes = pyramid_debugtoolbar pyramid_tm
pyramid.reload_templates = false
sqlalchemy.url = %DB_URL%
[server:main]
use = egg:waitress#main
listen = *:6543
[pipeline:main] # WSGI middleware: [filter:x] + [app:y]
pipeline = translogger bookshelf

reload_templates reload_assets reload_all
debug_authorization debug_notfound debug_routematch
debug_all prevent_http_cache prevent_cachebust
default_locale_name csrf_trusted_origins pyramid.tweens.
Env override: PYRAMID_RELOAD_TEMPLATES=1 (an empty var means
false). Custom: asbool() aslist(); secrets from the environment,
never from git.
```

URL DISPATCH

```
config.add_route("book", "/books/{id:d+}")
config.add_route("files", "/f/{subpath}") # remainder tuple
config.add_route("api", "/api/books", request_method="GET",
    factory=BookFactory)
request.matchdict["id"]; request.matched_route.name
request.route_url("book", id=3); request.route_path(...)
config.add_static_view("static", "pkg:static")

Routes are tried in declaration order, first match wins. Predicates:
request_method request_param accept header match_param.
route_prefix, inherit_slash=True. Debug: proutes,
debug_routematch. Slash redirect:
notfound_view_config(append_slash=True).
```

TRAVERSAL & RESOURCES

```
class Shelf(dict):
    __name__ = __parent__ = None
    __acl__ = [(Allow, "owner:ada", "edit")]
    config = Configurator(root_factory=Root)
    /shelves/ada/dune/edit → context=Book("dune"),
        view_name="edit", subpath=()
    request.resource_url(context)
    @view_config(context=Book, name="edit", permission="edit")
    config.add_route("h", "/h/*traverse", factory=Root) # hybrid

Walk __getitem__ segment by segment; the first miss becomes
view_name, the rest subpath. Set __name__ / __parent__ (location
awareness) for resource_url and ACL inheritance.
```

VIEWS & PREDICATES

```
@view_defaults(route_name="book", renderer="json")
class BookViews:
    def __init__(self, request): self.request = request
    @view_config(request_method="GET")
    def get(self): return {...}
    @view_config(request_method="PUT", permission="edit",
        require_csrf=True)

Predicates: route_name request_method request_param
match_param accept header xhr context name containment
is_authenticated not(); custom: add_view_predicate. Options:
permission renderer http_cache require_csrf decorator
wrapper. Most specific predicate set wins; pvIEWS shows the lookup.
```

RENDERERS & JSON

```
@view_config(route_name="b", renderer="json") # string|jsonp
def b(request): return Book(...)
class Book:
    def __json__(self, request): return {"id": self.id}
json = JSON() # add_renderer("json", json)
json.add_adapter(date, lambda o, r: o.isoformat())
request.response.status = 201 # tweak the response
return Response(...) # bypasses the renderer
@view_config(renderer="templates/x.jinja2") # pyramid_jinja2
render_to_response("pkg:templates/x.jinja2", {...}, request)

Templates: asset specs pkg:path; BeforeRender adds globals; Jinja2
autoescapes; route_url/static_url filters.
```

REQUEST, RESPONSE & ERRORS

```
request.params | GET | POST | json_body | cookies | headers
config.add_request_method(get_user, "user", reify=True)
request.add_response_callback(cb); add_finished_callback(cb)
return HTTPFound(request.route_url("home")) # or raise
raise HTTPNotFound() # pyramid.httpexceptions
@exception_view_config(ValueError, renderer="json")
@notfound_view_config(renderer="404.jinja2")
@forbidden_view_config() # exception_only=True

WebOb-based Request/Response; FileResponse;
http_cache=3600. Exception views run after pyramid_tm aborts.
debug_notfound explains 404s.
```

STATIC ASSETS & CACHE BUSTING

```
config.add_static_view("static", "pkg:static",
    cache_max_age=3600)
request.static_url("pkg:static/app.css") # also static_path
config.add_cache_buster("pkg:static",
    QueryStringConstant(CacheBuster("v7")) # or Manifest...
config.add_static_view("s", "pkg:s",
    content_encodings=["gzip"]) # 2.0
config.override_asset("pkg:static/", "other:static/")
```

SESSIONS & FLASH

```
factory = SignedCookieSessionFactory(
    os.environ["SESSION_KEY"], httponly=True, secure=True,
    samesite="Lax", timeout=3600)
config.set_session_factory(factory)
request.session["k"] = "v"; session.changed()
request.session.invalidate() # log out / reset
request.session.flash("Saved", "ok") # queue "ok"
request.session.pop_flash("ok") # peek_flash: no pop

Cookie is signed, not encrypted, about 4 KB, JSON-only since 2.0.
Without a factory request.session raises AttributeError. Server-
side: pyramid_redis_sessions, pyramid_nacl_session.
```

2.0 SECURITY POLICY

```
class Policy:
    def __init__(self, secret): # ISecurityPolicy
        self.h = AuthTktCookieHelper(secret, hashlib="sha512")
    def identity(self, request): # user or None
    def authenticated_userid(self, request): # ...
    def permits(self, request, context, permission):
        return ACLHelper().permits(
            context, principals, permission)
    def remember(self, request, userid, **kw): ...
    def forget(self, request, **kw): ...
    config.set_security_policy(Policy(os.environ["AUTH_KEY"]))
    headers = remember(request, user.id) # pyramid.security
    return HTTPFound("/", headers=headers) # forget() to log out

request.identity, is_authenticated predicate. Load the user once
with RequestLocalCache or a reified method. Helpers:
SessionAuthenticationHelper, extract_http_basic_credentials.
Hash passwords with argon2-cffi/bcrypt.
```

ACLS & PERMISSIONS

```
from pyramid.authorization import (Allow, Deny, Everyone,
    Authenticated, ALL_PERMISSIONS, DENY_ALL, ACLHelper)
class Shelf:
    __acl__ = [(Allow, Everyone, "view"),
        (Allow, "owner:ada", ALL_PERMISSIONS),
        DENY_ALL]
    config.set_default_permission("view")
    @view_config(permission=NO_PERMISSION_REQUIRED) # public
    request.has_permission("edit", shelf)

ACE order matters: first match wins; lookup walks up __parent__
until an ACL decides. URL dispatch: route factory=.
debug_authorization logs decisions.
```

CSRF

```
config.set_csrf_storage_policy(
    CookieCSRFStoragePolicy()) # or Session...
config.set_default_csrf_options(require_csrf=True,
    header="X-CSRF-Token", allow_no_origin=False)
def csrf_token(request); new_csrf_token(request)
check_csrf_token(request) # manual
@view_config(require_csrf=False) # opt out

Default token field csrf_token. On HTTPS the Origin/Referer must
match host or csrf_trusted_origins; check_origin=False disables.
Forms: {{ get_csrf_token() }} in templates. JSON/SPA: send the
header. Not a CORS fix.
```

FORMS: DEFORM & COLANDER

```
class BookSchema(colander.MappingSchema):
    title = colander.SchemaNode(colander.String(),
        validator=colander.Length(min=1))
    form = deform.Form(BookSchema(), buttons=("save",))
    try: appstruct = form.validate(request.POST.items())
    except deform.ValidationFailure as e:
        return ("form": e.render())
    return {"form": form.render()}
```

PYRAMID_TM + SQLAlchemy + ALEMBIC

```
config.include("pyramid_tm"); config.include("pyramid_retry")
retry.attempts = 3 # [app:main]; TransientError retries
class Book(Base): # SQLAlchemy 2.0 declarative
    id: Mapped[int] = mapped_column(primary_key=True)
request.dbsession.add(Book(...)) # zope.sqlalchemy joins tm
request.tm.doom() # force abort tm.commit_veto = ...
alembic -c development.ini revision --autogenerate -m "x"
alembic -c development.ini upgrade head
initialise_db development.ini # cookiecutter script

One transaction per request: commit on success, abort on exception. Do
not commit() yourself; mark_changed for raw SQL. ZODB backend:
pyramid_zodbconn, request.root. Redis/Mongo:
add_request_method + finished callback.
```

EVENTS, TWEENS & VIEW DRIVERS

```
@subscriber(NewRequest) # ContextFound, BeforeTraversal,
def on_req(event): ... # NewResponse, BeforeRender
def timing_factory(handler, registry):
    def tween(request):
        t = time.time(); resp = handler(request)
        resp.headers["X-Time"] = str(time.time() - t)
        return resp
    return tween
config.add_tween("pkg.timing_factory", over=EXCVIEW)
# INGRESS | MAIN | under= also
config.add_view_driver(deriver, name="x") # ptweens
config.add_subscriber(fn, NewRequest)

Order: tweens (INGRESS → ... EXCVIEW ... MAIN) wrap routing,
events, drivers, view. Avoid get_current_request().
```

I18N

```
- = TranslationStringFactory("bookshelf")
request.localizer.translate_("Hello")
request.localizer.pluralize(5, p, n, domain="bookshelf")
config.add_translation_dirs("bookshelf:locale")
pot-create -o locale/bookshelf.pot bookshelf # lingua
msgfmt / pybabel compile -d locale
default_locale_name = en # config.set_locale_negotiator(f)
```

P-SCRIPTS

```
pserve development.ini --reload pshell development.ini
proutes development.ini pvIEWS development.ini /path
ptweens development.ini prequest development.ini /
pdistrreport
env = bootstrap("development.ini") # env["request"]
env["__closer"]() # in a script
setup_logging("development.ini") # pyramid.paster
```

TESTING

```
with testing.testConfig(settings={...}) as config:
    resp = my_view(request.DummyRequest(params={"q": "x"}))
    config.testing_securitypolicy(user="ada", permission=True)
    testapp = TestApp(app) # WebTest, functional
    res = testapp.get("/", status=200); res.json; res.form;
    testapp.post_json("/api/books", {...}, status=201)
    @pytest.fixture # conftest: testing.ini, app, testapp,
    def dbsession(...) # rollback per test pytest --cov
```

DEPLOYMENT & LOGGING

```
pserve production.ini
unicorn --paste production.ini --workers 4 # Linux
[server:main] use = egg:waitress#main
trusted_proxy = 127.0.0.1 # honour X-Forwarded-*
location / { proxy_pass http://127.0.0.1:6543; } # nginx

nginx terminates TLS, forwards X-Forwarded-*; Waitress
trusted_proxy* honours them. Pref mounts: vhosting. Production:
no debugtoolbar (intercept_exc shows source), reload off, logging
from [loggers], TransLogger, Sentry/OpenTelemetry WSGI.
```

EXTENDING & ADVANCED

```
def includeme(config): # add-on = package + includeme
    config.add_directive("add_thing", add_thing)
    config.include("addon"); config.override_asset("a:x", "b:y")
    sub = Request.blank("/api/books/1") # subrequest
    resp = request.invoke_subrequest(sub, use_tweens=True)
    cache = RequestLocalCache(load_user) # per-request cache
    user = cache.get_or_create(request)

Add-on catalogue: trypyramid.com; config.registry is the ZCA
registry (hook_zca); prevent_http_cache disables caching;
pdistrreport lists versions.
```

CHANGED SINCE OLDER TUTORIALS / BOOKS

2.0 set_authentication_policy + set_authorization_policy → one
set_security_policy() (ISecurityPolicy);
AuthTktAuthenticationPolicy → AuthTktCookieHelper,
ACLAuthenticationPolicy → ACLHelper
2.0 Allow, Deny, Everyone, Authenticated, ALL_PERMISSIONS,
DENY_ALL moved from pyramid.security to pyramid.authorization
2.0 effective_principals, unauthenticated_userid deprecated →
request.identity, is_authenticated

2.0 signed-cookie sessions use **JSON, not pickle**: values must be JSON-
serializable
2.0 pcreate scaffolds removed → cookiecutter; python setup.py
develop → pip install -e ".[testing]"
2.0 new: inherit_slash, RequestLocalCache, static
content_encodings, CSRF allow_no_origin / check_origin; Python
2 dropped

1.5 custom_predicates deprecated → add_view_predicate (not_()
for negation)
2.1 (2026-03) Python 3.10-3.14 only; HTTPInATeapot; SHA-256 in the
view system; lgettext removed
2.1 tutorials use pyproject.toml and SQLAlchemy 2.0; pkg_resources
still imported, so setuptools<82; still WSGI-only, no async views